Report Project: Conway's Game of Life

Ybe Vandamme, Jarno De Jonckheere

13 December 2024



1 Introduction

For the project, we chose to make an implementation of Game of Life. Game of Life is an automated game invented by John Conway. The design is quite simple. It consists of a grid with cells that are alive or dead. Each cell evolves according to a set of simple rules based on the state of its neighbors. It's a simple setup but an excellent case study for parallel computation.

1.1 Rules of Game of Life

The grid evolves following these rules:

1. Any live cell with fewer than two live neighbors dies, as if by underpopulation.
2. Any live cell with two or three live neighbors lives on to the next generation.
3. Any live cell with more than three live neighbors dies, as if by overpopulation.
4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

These rules are applied simultaneously to every cell on the grid, resulting in a new grid state or generation.

2 Why using GPU?

GPUs are designed to do parallel computations, in this case, it would be perfect. Each cell's next state could be computed in parallel. This would decrease the computation time for the entire grid.

3 First version of the implementation

In the initial version of the kernel (named "gameoflife_noStride_Synchronous") we mapped the 2D matrix of threads and cells to a 1D-array. After which for each cell the amount of live neighbors is counted. After this each cell is updated to it's next state.

```
1
2  __global__ void gameOfLifeKernel(int* d_current, int* d_next
       , int width, int height, int generation){
3      int x = blockIdx.x * blockDim.x + threadIdx.x;
4      int y = blockIdx.y * blockDim.y + threadIdx.y;
5
6      if (x >= width || y >= height){
7          return;
```

```
    }

    int index = y * width + x; //Cell selection
    int LiveNeighbors = 0;

    for (int dy = -1; dy <= 1; dy++){ //counts live
        neighbors
        for (int dx = -1; dx <=1; dx++){
            if (dx == 0 && dy == 0 ) continue;
            int nx = x + dx;
            int ny = y + dy;

            if(nx >= 0 && nx <width && ny >= 0 && ny <
                height){
                LiveNeighbors += d_current[ny * width + nx];
            }
        }
    }

    if (d_current[index] == 1){ // implements game-rules
        d_next[index] = (LiveNeighbors == 2 || LiveNeighbors
            == 3) ? 1 : 0;
    } else {
        d_next[index] = (LiveNeighbors==3) ? 1 : 0;
    }
```

Listing 1: gameOfLifeKernel

Then we also added two other functions: one to detect if there are still any cell's alive and one to detect if the grid still go's through changes. If we do not do this the simulation would be running indefinitely. (The simulation could still continue indefinitely but we added a maximum amount of generations in the while-loop in the main().)

```
__global__ void checkNoChangeKernel(int* d_current, int*
    d_next, int width, int height, int* d_noChange){
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y>= height) return;

    int index = y * width + x;

    if(d_current[index] != d_next[index]){
        *d_noChange = 0;
    }
}
```

Listing 2: checkNoChangeKernel

```
1
2  __global__ void checkAllDeadKernel(int* grid, int width, int
       height, int* d_allDead){
3      int x = blockIdx.x * blockDim.x + threadIdx.x;
4      int y = blockIdx.y * blockDim.y + threadIdx.y;
5
6      if (x >= width || y >= height) return;
7
8      int index = y * width + x;
9
10     if(grid[index] == 1){
11         *d_allDead = 0;
12     }
13 }
```

Listing 3: checkAlDeadKernel

4 Second version of the implementation

In this second version of the implementation (named "gameoflife_noStride _Asynchronous") we tried to speed up the execution time by cleaning up the code and adding asynchronous memory access and GPU/CPU execution. Here in this application it was mainly the removal of the unnecessary synchronization steps that caused the biggest boost in performance (as can be seen in figure 1) as there were not many kernels or pieces of CPU-code that could operate parallelly. (this shows how important it is to not have unnecessary synchronization steps.)

```
1      int max_generations = 500;
2      int generation= 0;
3
4      while(generation<max_generations){
5          int allDead = 1, noChange = 1;
6          //for timing purposes
7          clock_t begin = clock();
8          cudaMemcpy(d_allDead, &allDead, sizeof(int),
               cudaMemcpyHostToDevice);
9          cudaMemcpy(d_noChange, &noChange, sizeof(int),
               cudaMemcpyHostToDevice);
10
11         gameOfLifeKernel<<<gridDim, blockDim>>>(d_current,
               d_next, WIDTH, HEIGHT, generation); //updates to
               the next generation
12         cudaDeviceSynchronize();
13         // this can be done asynchronous?
14
15         //copy this generations output to CPU
16         cudaMemcpy(h_current, d_next,size,
               cudaMemcpyDeviceToHost);
```

```
          //write to the file
        for(int i =0;i<WIDTH*HEIGHT;i++){
            outPutFile<<h_current[i]<<";";
        }
        outPutFile<<std::endl;

        checkAllDeadKernel<<<gridDim, blockDim>>>(d_next,
            WIDTH, HEIGHT, d_allDead);
        cudaDeviceSynchronize();
        cudaMemcpy(&allDead, d_allDead, sizeof(int),
            cudaMemcpyDeviceToHost);

        checkNoChangeKernel<<<gridDim, blockDim>>>(d_current
            , d_next, WIDTH, HEIGHT, d_noChange);
        cudaDeviceSynchronize();
        cudaMemcpy(&noChange, d_noChange, sizeof(int),
            cudaMemcpyDeviceToHost);

        if(allDead){
            //printf("Simulation stopped: All cells are dead
                at generation %d.\n", generation);
            break;
        }

        if(noChange){
            //printf("Simulation stopped: No changes
                detected at generation %d.\n", generation);
            break;
        }

        int* temp = d_current;
        d_current = d_next;
        d_next = temp;

        generation++;
        clock_t end = clock();
        double time = (double) (end-begin) / CLOCKS_PER_SEC;
             //generation time
        timingFile<<"generation_time;"<<time<<std::endl;

    }
```

Listing 4: Update-loop of the first version

```
    int max_generations = 500;
    int generation= 0;

    while(generation<max_generations){
        int allDead = 1, noChange = 1;
        //for timing purposes
```

```
        clock_t begin = clock();
        cudaMemcpyAsync(d_allDead, &allDead, sizeof(int),
            cudaMemcpyHostToDevice,stream1);
        cudaMemcpyAsync(d_noChange, &noChange, sizeof(int),
            cudaMemcpyHostToDevice,stream1);

        gameOfLifeKernel<<<gridDim, blockDim,0,stream1>>>(
            d_current, d_next, WIDTH, HEIGHT, generation); //
            updates to the next generation
        //cudaDeviceSynchronize(); not nesecarry because
            cudamemcpy synchronize

        //copy this generations output to CPU
        cudaMemcpyAsync(h_current, d_next,size,
            cudaMemcpyDeviceToHost,stream1);


        checkAllDeadKernel<<<gridDim, blockDim,0,stream1>>>(
            d_next, WIDTH, HEIGHT, d_allDead);
        checkNoChangeKernel<<<gridDim, blockDim,0,stream1
            >>>(d_current, d_next, WIDTH, HEIGHT, d_noChange)
            ;

        cudaMemcpyAsync(&allDead, d_allDead, sizeof(int),
            cudaMemcpyDeviceToHost,stream1);
        cudaMemcpyAsync(&noChange, d_noChange, sizeof(int),
            cudaMemcpyDeviceToHost,stream1);

        //write to the file
        cudaStreamSynchronize(stream1);
        for(int i =0;i<WIDTH*HEIGHT;i++){
            outPutFile<<h_current[i]<<";";
        }
        outPutFile<<std::endl;

        if(allDead){
            //printf("Simulation stopped: All cells are dead
                at generation %d.\n", generation);
            break;
        }

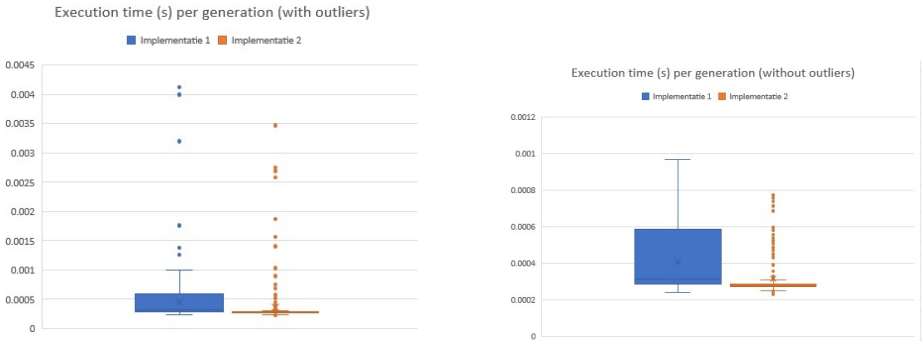
        if(noChange){
            //printf("Simulation stopped: No changes
                detected at generation %d.\n", generation);
            break;
        }

        int* temp = d_current;
        d_current = d_next;
        d_next = temp;
```

```
        generation++;
        clock_t end = clock();
        double time = (double) (end-begin) / CLOCKS_PER_SEC;
            //generation time
        timingFile<<"generation_time;"<<time<<std::endl;

    }
```

Listing 5: Update-loop of the second version



(a) Chart 1

(b) Chart 2

Figure 1: Timing-Charts

5 Third version of the implementation

In this last version of the implementation we added striding. In the report we will only show how striding was implemented in the gameOfLife-kernel but it was implemented in a similar method in the other two kernels. The implementation works by mapping every thread to more to as many cells as is necessary to cover the whole matrix. Because we are working with a 2D-matrix of threads this requires a nested for-loop.

```

__global__ void gameOfLifeKernel(int* d_current, int* d_next
    , int width, int height, int generation){
        int stride_x = gridDim.x * blockDim.x; //stride in x
            -direction
        int stride_y = gridDim.y * blockDim.y; //stride in y
            -direcion
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height){
```

```
9           return;
10      }
11
12      for(int i = x;i<width;i+=stride_x){
13          for(int j = y;j<height;j+=stride_y){
14              int index = j * width + i; //Cell selection
15              int LiveNeighbors = 0;
16
17              for (int dy = -1; dy <= 1; dy++){ //counts live
                    neighbors
18                  for (int dx = -1; dx <=1; dx++){
19                      if (dx == 0 && dy == 0 ) continue;
20                      int nx = i + dx;
21                      int ny = j + dy;
22
23                      if(nx >= 0 && nx <width && ny >= 0 && ny
                            < height){
24                          LiveNeighbors += d_current[ny *
                                width + nx];
25                      }
26                  }
27              }
28
29              if (d_current[index] == 1){ // implements game-
                    rules
30                  d_next[index] = (LiveNeighbors == 2 ||
                        LiveNeighbors == 3) ? 1 : 0;
31              } else {
32                  d_next[index] = (LiveNeighbors==3) ? 1 : 0;
33              }
34          }
35      }
```

Listing 6: GameOfLife-kernel with striding

6 Visualisation

To visualise the execution of the program we chose to first write the output of the cuda-program to a file after which we read the file in a python program and visualised the execution using the tkinter-library. The result looked as follows can be seen in figure 2

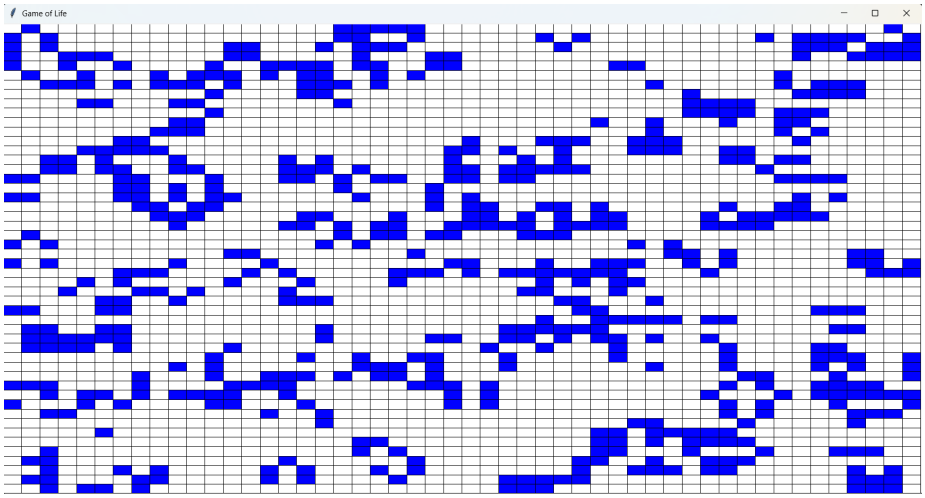


Figure 2: Visualisation

For a clearer image of the visualisation of the program, an mp4-file can be found in the github repository.

7 Links to code

All the code you can find here: `https://github.com/JarnoDeJonckheere/GameOfLife/tree/main`.